AUTD3 触覚ボード
リファレンスマニュアル
Rust 編

版	日付	内容
1.00	2026/05/13	初版
1.10	2026/11/10	修正

変更履歴

版数	発行日	改版内容
1.00	2026/05/13	初版
1.10	2026/11/10	2.2 使用方法 確実にオフライン環境(キャッシュ優先)でビルドされるよう ビルド時のコマンドに「--offline」オプションを追加。

AUTD3 触覚ボード

リファレンスマニュアル Rust 編

目次

1. 概要 2
2. インストール 2
   2.1. クレート一覧及び共通事項 2
   2.2. 使用方法 3
3. 制御手順 4
   3.1. 基本構造 4
   3.2. モジュール構成とデータの流れ 5
   3.3. 操作フロー 6
4. クラス/メソッド詳細 7
   4.1. クラス/メソッド一覧 7
   4.2. Controller クラス 9
   4.3. AUTD3 クラス 13
   4.4. Geometry クラス/Device クラス/Transducer クラス 14
   4.5. Silencer クラス 18
   4.6. Focus クラス/FocusOption クラス 19
   4.7. Plane クラス/PlaneOption クラス 20
   4.8. Bessel クラス/BesselOption クラス 21
   4.9. FociSTM クラス 22
   4.10. GainSTM クラス/GainSTMOption クラス 23
   4.11. Null クラス 24
   4.12. Sine クラス/SineOption クラス 25
   4.13. Square クラス/SquareOption クラス 27
   4.14. Static クラス 28
   4.15. EtherCrab クラス/EtherCrabOption クラス 29

1. 概要

本書は、(空中触覚用超音波フェーズドアレイデバイス(以下、AUTD3 デバイス)を
Rust を用いて動作させる際の手順及び、ライブラリ仕様について記述する。

2. インストール

2.1. クレート一覧及び共通事項

・ Rust から AUTD3 デバイスを稼働させるためのクレート(ライブラリ)を下記に示す。

パッケージ名	内容等
autd3	AUTD3 デバイスを制御する基本クレート
autd3-gain-holo	複数焦点に関する機能を提供するクレート
autd3-link-twincat	TWinCAT を用いて AUTD3 デバイスと通信する為のクレート
autd3-link-ethercrab	EtherCrab を用いて AUTD3 デバイスと通信する為のクレート
autd3-link-remote	リモートサーバ又はシミュレータに接続する際に使用するクレート
autd3-link-soem	SOME を用いて AUTD3 デバイスと通信する為のクレート
autd3-emulator	エミュレータと通信する為のクレート

入門キットでは autd3 / autd3-gain-holo / autd3-link-ethercrab のみ利用可能となっている。

・ 「autd3-link-ethercrab」は EtherCrab 経由で AUTD3 デバイスと通信する際にのみ必要となる。

・ 上記クレートは crates.io に登録されており、cargo を用いて使用することができる。

2.2. 使用方法

・ cargo を用いて各プロジェクトにクレートを追加することで利用できる。

例）

```
$ cargo new --bin sample01                                  プロジェクト作成
$ cd sample01                                               プロジェクトフォルダに移動
$ cp ~/Desktop/RustSample/Sample01_single.rs src/main.rs    サンプルコードのコピー
$ cargo add autd3                                           autd3 ライブラリ追加 ※
$ cargo add autd3-link-ethercrab                            ethercrab ライブラリ追加 ※
$ cargo build                                               ビルド
$ sudo ./target/debug/sample01                              実行
```

※ 入門キットなど、事前にダウンロードされておりインターネットに接続されていない環境では
下記の様に「--offline」オプションを指定する。

```
$ cargo add autd3 --offline                                 autd3 ライブラリ追加
$ cargo add autd3-link-ethercrab --offline                  ethercrab ライブラリ追加
$ cargo build --offline                                     ビルド
```

・ 入門キットでは下記クレートが事前ダウンロードされており、
インターネットに接続することなく使用できる。

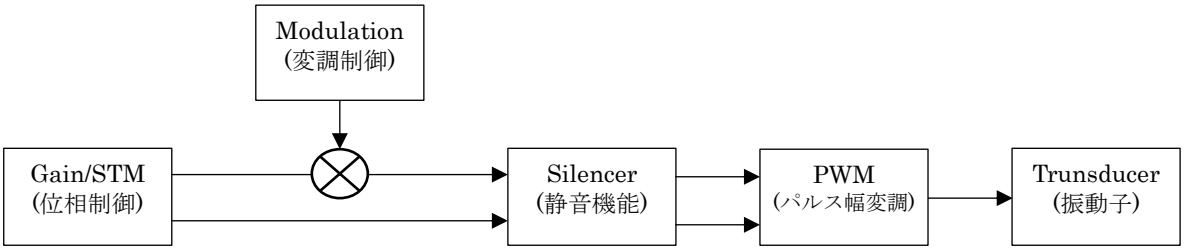
```
autd3
autd3-gain-holo
autd3-link-ethercrab
```

ただし、自動的にインターネット経由でダウンロードが行われないようにするため、
「--offline」オプションを指定する必要がある。

3. 制御手順

3.1. 基本構造

・ AUTD3 デバイスが動作する際の概念図を下記に示す。

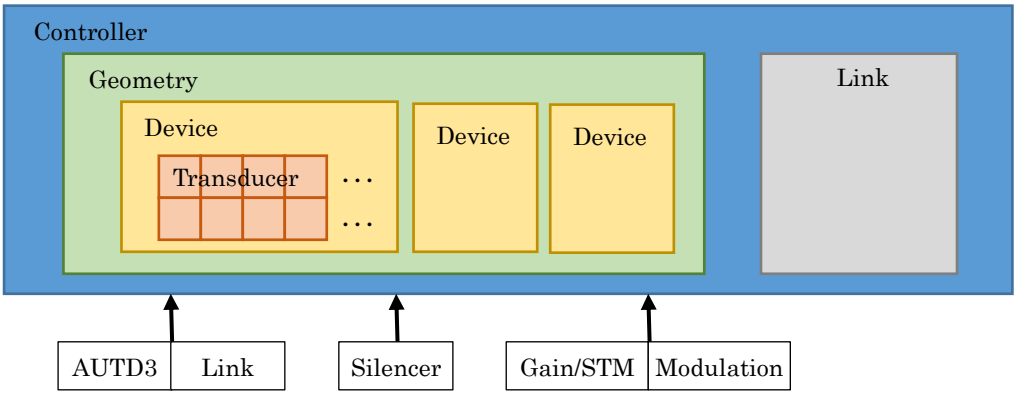


Gain : 位相/振幅を管理する。
STM : Spatio-Temporal Modulation(時空間変調)機能で、位相/振幅を時間列で管理する。
Modulation : AM 変調を管理する。
Silencer : 静音化処理を管理する。
PWM : 上記データに基づき、各振動子を駆動させる電気信号を生成する。
Trunsducer : 電気信号により超音波を発生させる。

・ AUTD3 ライブラリの主な構成要素を下記に示す。
(上記動作概念図中の要素と同名のクラスは、その要素の動作や状態等を管理する)

コンポーネント	概要
Controller	AUTD3 デバイスに対する接続/指示送信などの操作を行う。
AUTD3	個々の AUTD3 デバイス情報を管理する。
Geometry	AUTD3 デバイス群を管理する。 (Geometry クラスは Device クラスのコンテナであり、 Device クラスは Transducer クラスのコンテナ)
Link※1	AUTD3 デバイスとの通信を管理する。 TwinCAT 接続(TwinCAT クラス)、EtherCrab 接続(EtherCrab クラス)などがある。
Gain	位相/振幅を管理する。
STM	Spatio-Temporal Modulation(時空間変調)機能で、位相/振幅を時間列で管理する。
Modulation※1	AM 変調処理を管理する。 正弦波(Sine クラス)、矩形波(Square クラス)などがある。
Silencer	静音化処理を管理する。

※1 実際にはこれらのクラスを継承したクラスを使用する。

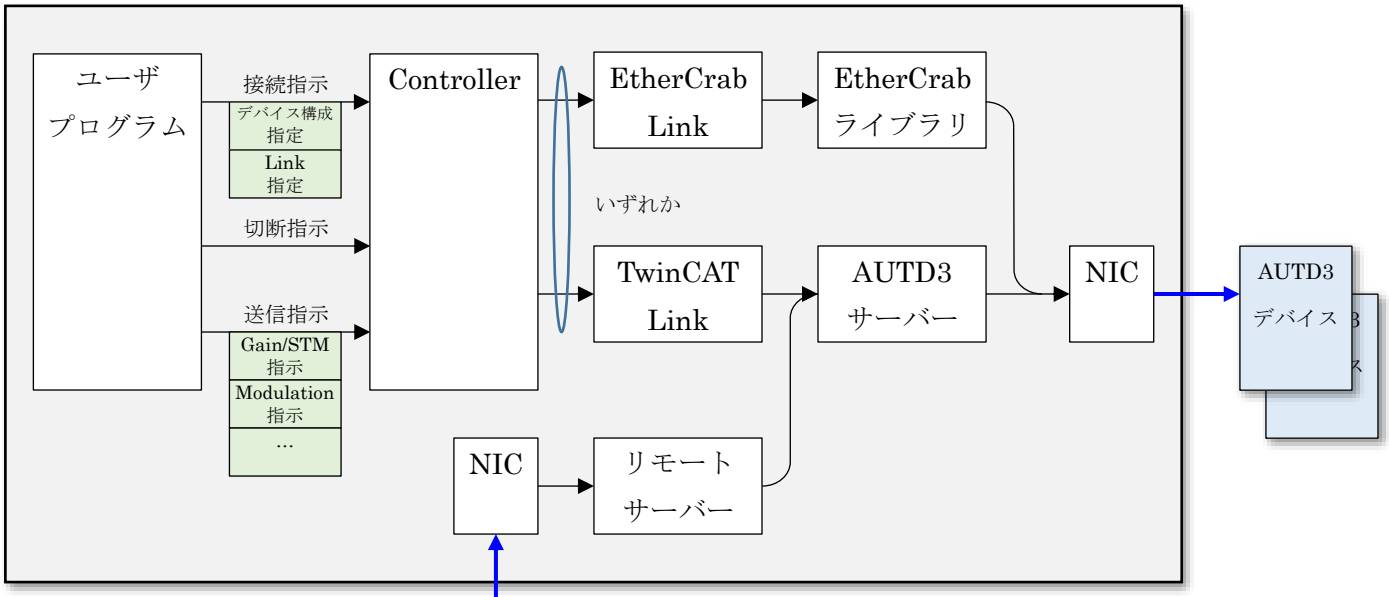


参考：https://shinolab.github.io/autd3-doc/jp/Users_Manual/tutorial/concept.html

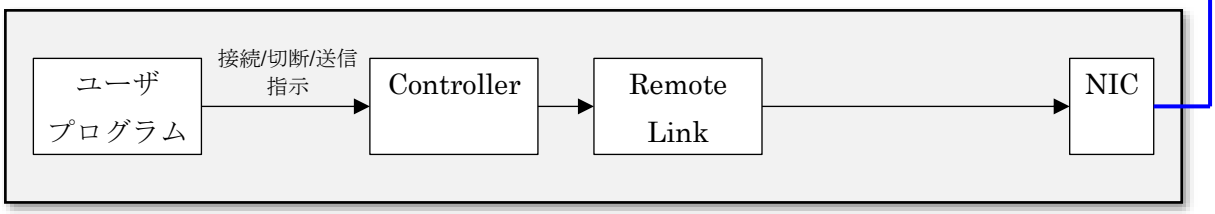
3.2. モジュール構成とデータの流れ

・ 各モジュールの関連とデータの流れの概要を下記に示す。

AUTD3 デバイス(触覚ボード)が接続されている PC



AUTD3 デバイス(触覚ボード)が接続されていない PC



・ ユーザプログラムコードは「Controller」に対してデバイスを動作させる各種操作を指示し、「Controller」は各種「Link」経由でデバイスとの通信を行う。

・ Rust 用として下記の「Link」が提供されている。

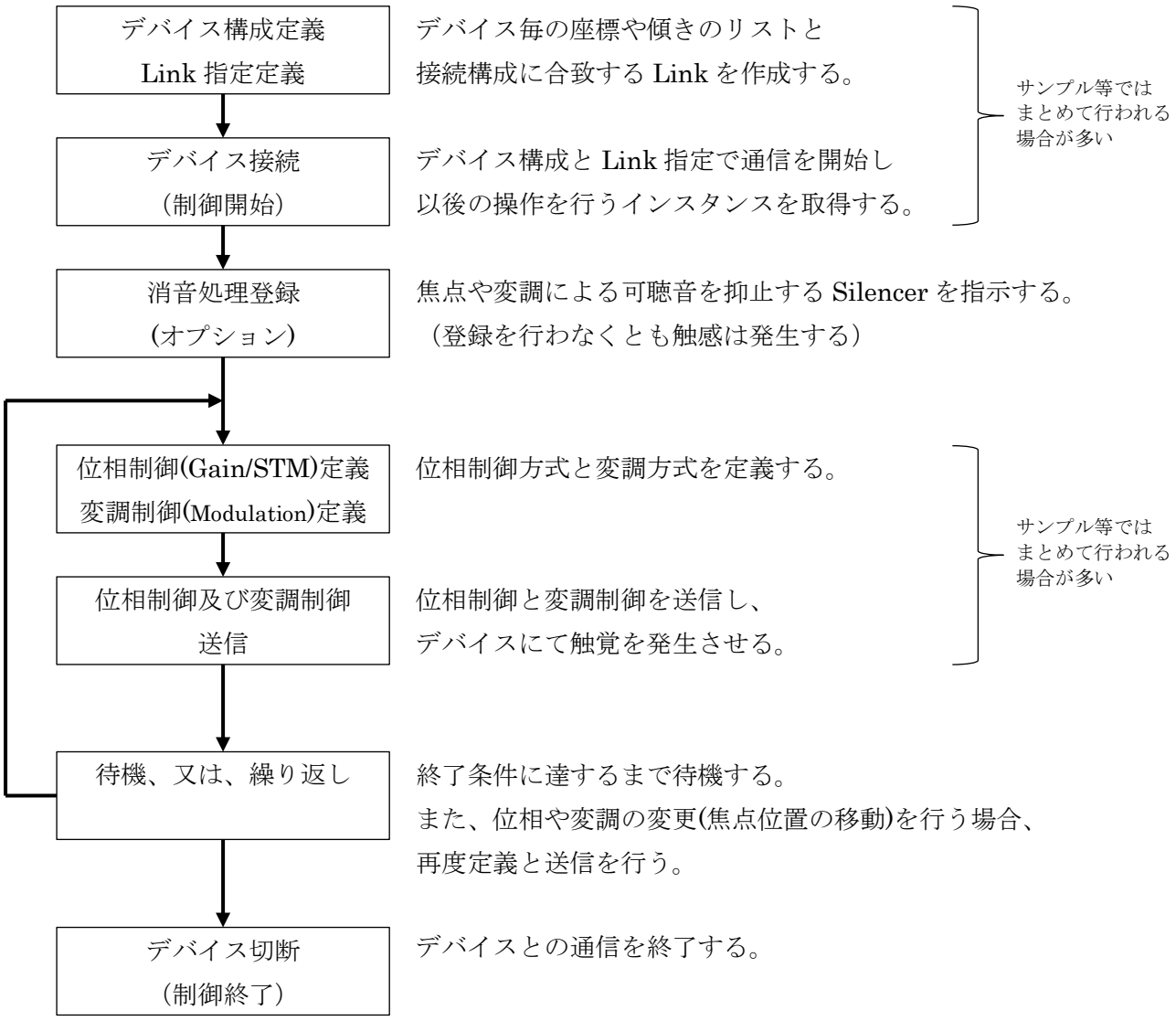
Link 名	内容
TwinCAT Link	AUTD3 サーバー(TwinCAT ライブラリ)経由でデバイスと通信
EtherCrab Link	EtherCrab 経由でデバイスと通信
SOEM Link	SOEM(Simple Open Source EtherCAT Master) 経由でデバイスと通信
Remote Link	リモートサーバ

・ 「リモートサーバー」を用いる場合、デバイスが接続されていない PC から各種制御を行うこともできる。

・ ユーザプログラムは位相制御(Grain/STM)や変調制御(Modulation)等の指示(Datagram を継承したクラス)を Controller 経由で AUTD3 デバイスに送信し、デバイスの動作を決定する。

3.3. 操作フロー

・ ユーザプログラムでボードを使用する際の手順を下記に示す。



・ 位相制御および変調制御を送信することにより触感が継続して生成される。
送信処理はブロックせずにユーザコードに戻る為、必要に応じて焦点の変更や待機を行う。
（ユーザアプリが終了すると触感生成も終了する為、触感生成を継続させるには待機等が必要）

4. クラス/メソッド詳細

4.1. クラス/メソッド一覧

・ AUTD3 用ライブラリで提供されているクラス/メソッド/定数の一覧を下記に示す。

○autd3 クレート

クラス	メソッド/フィールド	概要
Controller	open	AUTD3 デバイスの接続
	open_with	オプションを指定しての AUTD3 デバイス接続
	sender	AUTD3 デバイス送信(リンク)取得
	sender_with_sleeper	待機機構を指定して AUTD3 デバイス送信取得
	send	AUTD3 デバイスへ送信
	inspect	検査結果取得
	close	AUTD3 デバイスの切断
	firmware_version	AUTD3 デバイスのファームウェアバージョン取得
	fpga_state	AUTD3 デバイスの fpga 状態取得
	into_boxed_link	ボックス化したリンクを取得
	from_boxed_link	ボックス化したリンクから Controller を取得
AUTD3		AUTD3 デバイス定義
Geometry	num_devices	構成内の AUTD3 デバイスの数取得
	num_transducers	構成内の振動子の数取得
	center	構成内の中心座標取得
	reconfigure	再構成
Device	idx	インデックス取得
	num_transducers	振動子の数取得
	rotation	回転情報取得
	center	デバイスの中心座標取得
	x_direction	X 方向取得
	y_direction	Y 方向取得
	axial_direction	軸方向種痘 k
Transducer	idx	インデックス取得
	dev_idx	所属する AUTD3 デバイスのインデックス取得
	position	位置取得
Focus	pos	焦点位置
	option	オプション
FocusOption	intensity	強度
	phase_offset	位相オフセット
Bessel	apex	ビームの頂点
	dir	ビームの方向
	theta	ビームに垂直な平面と仮想円数側面との角度
	option	オプション
BesselOption	intensity	強度
	phase_offset	位相オフセット
Plane	dir	方向
	option	オプション
PlaneOption	intensity	強度
	phase_offset	位相オフセット
FociSTM		１～8 個の単一焦点を生成する時空間変調(STM)
GainSTM		任意の Gain を扱う時空間変調(STM)
Null		なにも出力しない位相制御

クラス	メソッド/フィールド	概要
Sine	freq	周波数
	option	オプション
	into_nearest	出力できる最も近い周波数に変換
SineOption	intensity	強度
	offset	オフセット
	phase	位相
	clamp	クランプ設定
	sampling_config	サンプリング構成
Square	freq	周波数
	option	オプション
	into_nearest	出力できる最も近い周波数に変換
SquareOption	low	変調低値
	high	変調高値
	duty	デューティ比
	sampling_config	サンプリング構成
Static	intensity	強度

○autd3_link_ethercrab クレート

クラス	メソッド/フィールド	概要
EtherCrab	new	新規作成
	with_runtime	Tokio ランタイムを指定しての新規作成
	open	接続開始
	close	接続終了
	alloc_tx_buffer	送信バッファ確保
	send	データ送信
	receive	メッセージ受信
	is_open	接続状態取得
	ensure_is_open	接続状態確認
EtherCrabOption	ifname	ネットワークインターフェース名
	state_check_period	状態を確認する間隔
	sync0_period	sync0 期間
	sync_timeout	同期タイムアウト値
	sync_tolerance	同期許容時間

4.2. Controller クラス

(1) 概要

・ 全 AUTD3 デバイスに対する操作の指示、及び、現在の状態の提供等を行う。

参照：https://shinolab.github.io/autd3-doc/jp/Users_Manual/API/controller.html

(2) 生成

・ Controller インスタンスは、Controller の open メソッドにより作成する。

(3) open メソッド

```
pub fn open<D: Into<Device>, F: IntoIterator<Item = D>>( devices: F, link: L, )
        -> Result<Self, AUTDDriverError>
```

引数	内容
devices	構成する AUTD3 デバイスのリスト
link	使用するリンク
戻り値: Controller	Controller インスタンス

・ AUTD3 デバイスとの通信を開始し、戻り値として Controller インスタンスを返す。

・ 通常、引数 devices には AUTD3 型配列を指定する。

AUTD3 デバイスが 1 台構成の場合の例

```
# デバイスリスト
let devices = [ AUTD3 {
                    pos: Point3::origin(),
                    rot: UnitQuaternion::identity(),}
];

# 使用するリンク
let link = …

# デバイスへの接続(EtherCrab を使用)
let mut autd = Controller::open( devices, link )?;
```

(4) close メソッド

```
pub fn close(self) -> Result<(), AUTDDriverError>
```

引数/戻り値:型	内容
引数なし	
戻り値なし	

・ AUTD3 デバイスとの通信を終了する。

(5) send メソッド

```
pub fn send<'a, D: Datagram<'a>>( &'a mut self, s: D, ) -> Result<(), AUTDDriverError>
```

引数/戻り値:型	内容
s	送信するデータ(指示)
戻り値	エラー情報(エラー発生時)

・ AUTD3 デバイスにデータ(各種指示)を送信する。

・ 引数には下記の様なデータ(クラス)を指定する。

送信できるデータ(クラス)	内容
Bessel	ベッセルビーム Gain 指示
Focus	単焦点 Gain 指示
Group	AUTD3 デバイスのグループ設定
Null	ゼロ振幅 Gain 指示
Plane	平面波 Gain 指示
Silencer	サイレンサ指示
Uniform	全振動子への同一位相/振幅指示

・ 2 要素タプルにすることにより、二つのデータを同時に送信することができる。

```
// 単一焦点の位相制御指示を生成
let g = Focus {
        pos: autd.center() + Vector3::new( 0., 0., 150.0 * mm ),
        option: FocusOption::default()
};

// Sin 波形の変調制御指示を生成
let m = Sine {
        freq: 150 * Hz,
        option: SineOption::default()
};

// 位相制御と変調制御を送信(動作開始)
autd.send( (m, g) )?;
```

(6) firmware_version メソッド

```
pub fn firmware_version( &mut self, ) -> Result<Vec<FirmwareVersion>, AUTDDriverError>
```

引数/戻り値:型	内容
引数なし	
戻り値	AUTD3 デバイス毎のファームウェア情報

・ 戻り値として AUTD3 デバイスのファームウェアバージョンのリストを返す。

(7) sender メソッド

```
pub fn sender(&mut self, option: SenderOption) -> Sender<'_, L, StdSleeper>
```

引数/戻り値	内容
option	送信設定
戻り値	現在の link インスタンス

・ 指定された送信設定が適用された送信管理を返す。

・ 送信設定(SenderOption)には、下記を指定する。

引数	内容
send_interval	送信間隔
receive_interval	受信間隔
timeout	タイムアウト時間
parallel	パラレルモード

(8) center メソッド/num_deveices メソッド/num_transducers メソッド

```
pub fn num_devices(&self) -> usize
pub fn num_transducers(&self) -> usize
pub fn center(&self) -> OPoint<f32, Const<3>>
```

・ 現在のデバイス状態を返す。詳細は Geometry クラスの同名メソッドを参照。

(9) fpga_state メソッド

```
pub fn fpga_state(&mut self) -> Result<Vec<Option<FPGAState>>, AUTDDriverError>
```

引数/戻り値:型	内容
なし	
戻り値	FPGA ステータス

・ AUTD3 デバイス(FPGA)の状態を取得する。

・ このメソッドを呼び出す前に、ReadsFPGAState を送信し、状態取得を有効化しておく必要がある。

```
let mut autd = Controller::open([AUTD3::default()], Nop::new())?;
autd.send(ReadsFPGAState::new(|_| true))?;
let states = autd.fpga_state()?;
```

4.3. AUTD3 クラス

(1) 概要

・ 接続されている個々の AUTD3 デバイス情報を管理する。

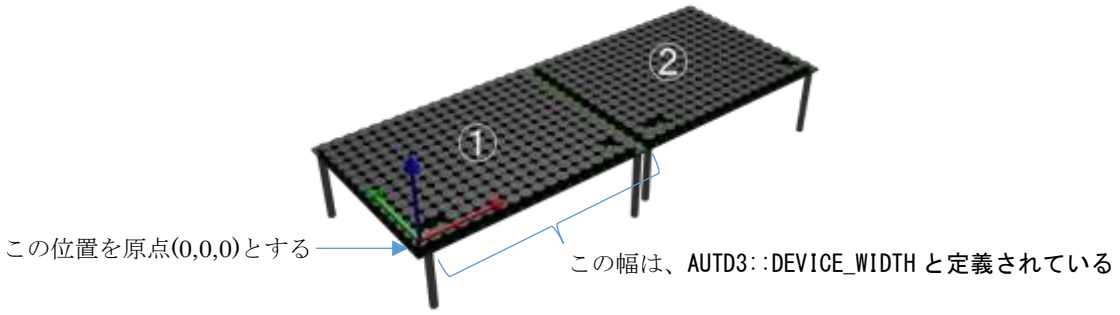
・ AUTD3 デバイスの物理的な設置状態に合わせて定義し、接続時に指定する。

(2) 作成

```
impl<R> AUTD3<R>
    where R: Into<Unit<Quaternion<f32>>> + Debug,
    pub fn new(pos: OPoint<f32, Const<3>>, rot: R) -> AUTD3<R>
```

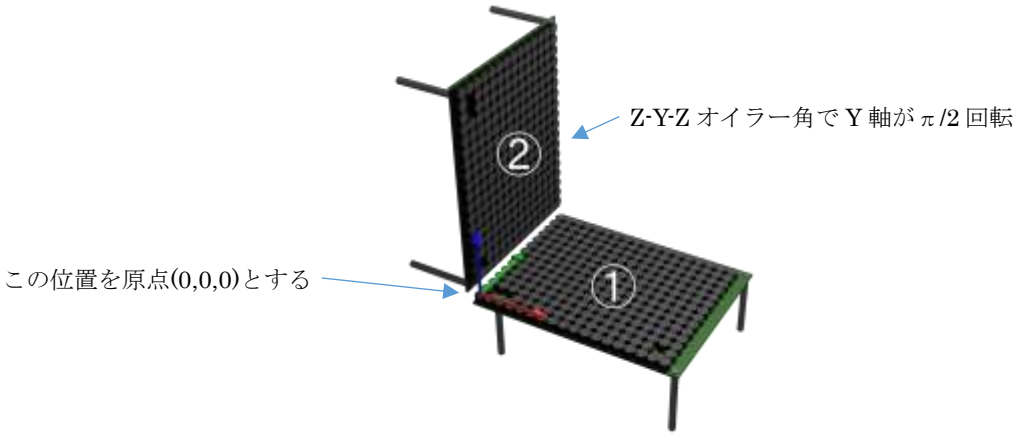
引数	内容
pos	AUTD3 ボードの位置(基準座標を[x,y,z]で指定)
rot	AUTD3 ボードの向き(ベクトルを[x,y,z,w]で指定)

・ 横に 2 台並べて設置する場合は、下記の様な定義となる。




```
① AUTD3{ pos: UnitQuaternion::identity(), rot: UnitQuaternion::identity() }
② AUTD3{ pos: Point3::new(AUTD3::DEVICE_WIDTH, 0., 0.), rot: UnitQuaternion::identity() } }
```

・ 下記の様に立体的に配置する場合、下記の様な定義となる。




```
① AUTD3{ pos: UnitQuaternion::identity(), rot: UnitQuaternion::identity()}
② AUTD3{ pos: Point3::new( 0., 0., AUTD3.DEVICE_WIDTH ],
          rot: EulerAngle::ZYZ(0. * rad, PI/2.0 * rad, 0. * rad).into()}
```

・ Controller クラスの open メソッドの第 1 引数に上記のリストを指定する。

参考：https://shinolab.github.io/autd3-doc/jp/Users_Manual/tutorial/multiple.html

4.4. Geometry クラス/Device クラス/Transducer クラス

(1) 概要

・ AUTD3 デバイスを構成する要素の状態等について管理する。
これらのクラスはユーザプログラムで生成することはなく、現在の状態を Controller クラスが提供する。

(2) center メソッド（Geometry クラス）

```
pub fn center(&self) -> OPoint<f32, Const<3>>
```

引数/戻り値:型	内容
引数なし	
戻り値	中心座標[x, y, z]

・ 戻り値として空間(全 AUTD3 デバイス)の中心位置を返す。

(3) num_devices メソッド（Geometry クラス）

```
pub fn num_devices(&self) -> usize
```

引数/戻り値:型	内容
引数なし	
戻り値	AUTD3 デバイスの数

・ 戻り値として AUTD3 デバイスの数を返す。

(4) num_transducers メソッド（Geometry クラス）

```
pub fn num_transducers(&self) -> usize
```

引数/戻り値:型	内容
引数なし	
戻り値	振動子の数

・ 戻り値として全振動子の数を返す。

(5) axial_direction メソッド（Device クラス）

```
pub const fn axial_direction(&self) -> UnitVector3
```

引数/戻り値:型	内容
引数なし	
戻り値	デバイスの軸方向ベクトル

・ デバイスの軸方向ベクトル(振動子が向く方向)を返す。

(6) center メソッド（Device クラス）

```
pub const fn center(&self) -> Point3
```

引数/戻り値:型	内容
引数なし	
戻り値	中心座標

・ 戻り値として、デバイスの中心位置を返す。
（Geometry の center メソッドは、全てのデバイスを合わせた空間の中心位置を返すが、
Device の center メソッドは、各デバイスの中心位置を返す）

(7) idx メソッド（Device クラス）

```
pub const fn idx(&self) -> usize
```

引数/戻り値:型	内容
引数なし	
戻り値	中心座標

・ 戻り値として、デバイスのインデックスを表す数値を返す。

(8) num_transducers メソッド（Device クラス）

```
pub const fn num_transducers(&self) -> usize
```

引数/戻り値:型	内容
引数なし	
戻り値	振動子の数を取得

・ 戻り値として、デバイス内の振動子数を返す。

(9) rotation メソッド（Device クラス）

```
pub const fn rotation(&self) -> UnitQuaternion
```

引数/戻り値:型	内容
引数なし	
戻り値	回転ベクトル

・ 戻り値として、デバイスの回転ベクトルを返す。

(10) x_direction メソッド/ y_direction メソッド（Device クラス）

```
pub const fn x_direction(&self) -> UnitVector3
pub const fn y_direction(&self) -> UnitVector3
```

引数/戻り値:型	内容
引数なし	
戻り値	デバイスの X/Y 方向ベクトル

・ 戻り値として、デバイスの X 方向ベクトル、又は、Y 方向ベクトルを返す。

(11) dev_idx メソッド（Transducer クラス）

```
pub const fn dev_idx(&self) -> usize
```

引数/戻り値:型	内容
引数なし	
戻り値	所属するデバイスのインデックス

・ 戻り値として、振動子が所属するデバイスのインデックスを返す。

(12) idx メソッド（Transducer クラス）

```
pub const fn idx(&self) -> usize
```

引数/戻り値:型	内容
引数なし	
戻り値	振動子のインデックス

・ 戻り値として、デバイス内の振動子インデックス値を返す。

(13) position メソッド（Transducer クラス）

```
pub const fn position(&self) -> Point3
```

引数/戻り値:型	内容
引数なし	
戻り値	振動子の位置

・ 戻り値として、振動子の位置を返す。

4.5. Silencer クラス

(1) 概要

・ 振幅変調に伴う可聴音の発生を抑える Silencer 機能の動作を指示する。

参照：https://shinolab.github.io/autd3-doc/jp/Users_Manual/API/silencer.html

(2) default メソッド

```
fn default() -> Silencer<FixedCompletionSteps>
```

・ デフォルトモード(FixedCompletionTime)の Silencer を返す。

(3) disable メソッド

```
impl Silencer<FixedCompletionSteps>
  pub const fn disable() -> Silencer<FixedCompletionSteps>
```

・ Silencer 動作を行わない Silencer を返す。

```
autd.send( Silencer::default() )?;    # Silencer 有効

autd.send( Silencer::disable() )?;    # Silencer 無効
```

4.6. Focus クラス/FocusOption クラス

(1) 概要

・ 単一焦点を生成する。
（空間上の一点に触覚を発生させる）

・ Gain クラスを継承する。

参照：https://shinolab.github.io/autd3-doc/jp/Users_Manual/API/gain/focus.html

(2) 生成

○Focus クラス

```
pub struct Focus {
    pub pos: Point3,
    pub option: FocusOption,
}
```

引数	内容
pos	焦点を発生させる座標 [x,y,z]
option	オプション

○FocusOption クラス

```
#[repr(C)]pub struct FocusOption {
    pub intensity: Intensity,
    pub phase_offset: Phase,
}
```

引数	内容
intensity	出力振幅
phase_offset	位相オフセット

4.7. Plane クラス/PlaneOption クラス

(1) 概要

・ 平面波を出力させる。
（一点ではなく全体に触覚を発生させる）

・ Gain クラスを継承する。

参照：https://shinolab.github.io/autd3-doc/jp/Users_Manual/API/gain/plane.html

(2) 生成

○Focus クラス

```
pub struct Plane {
    pub dir: UnitVector3,
    pub option: PlaneOption,
}
```

引数	内容
direction	平面波の方向(法線ベクトル)
option	オプション

○PlaneOption クラス

```
#[repr(C)]pub struct PlaneOption {
    pub intensity: Intensity,
    pub phase_offset: Phase,
}
```

引数	内容
intensity	出力振幅
phase_offset	位相オフセット

4.8. Bessel クラス/BesselOption クラス

(1) 概要

・ ベッセルビームを生成する。
（Focus の様な一点ではなく、直線状の触覚を発生させる）

・ Gain クラスを継承する。

参照： https://shinolab.github.io/autd3-doc/jp/Users_Manual/API/gain/bessel.html

(2) 生成

○Bessel クラス

```
pub struct Bessel {
    pub apex: Point3,
    pub dir: UnitVector3,
    pub theta: Angle,
    pub option: BesselOption,
}
```

引数	内容
apex: ArrayLike	ビーム仮想円錐の頂点（振動子面の位置）
direction: ArrayLike	ビームの方向（触覚を有無直線の向き）
option: PlaneOption	オプション

○BesselOption クラス

```
#[repr(C)]pub struct BesselOption {
    pub intensity: Intensity,
    pub phase_offset: Phase,
}
```

引数	内容
intensity: Intensity	出力振幅
phase_offset: Phase	位相オフセット

4.9. FociSTM クラス

(1) 概要

・ 1～8 個の単一焦点を生成する時空間変調(STM)を指示する。

・ Focus 相当の単一焦点の座標リストをあらかじめ指定しておき、
指定した周波数で順次焦点を発生させる。(リストで指定した座標での焦点発生を繰り返す)

参照：https://shinolab.github.io/autd3-doc/jp/Users_Manual/API/stm.html
https://shinolab.github.io/autd3-doc/jp/Users_Manual/API/stm/focus.html

(2) 生成

○FociSTM クラス

```
pub struct FociSTM<const N: usize, T, C>where
    T: FociSTMGenerator<N>,{
    pub foci: T,
    pub config: C,
}
```

引数	内容
foci	焦点のリスト
config	周期またはサンプリング設定

・ 複数の焦点を発生する場合は、ControlPoints で各焦点毎の焦点リストを指定する。

4.10. GainSTM クラス/GainSTMOption クラス

(1) 概要

・ 任意の Gain を扱う時空間変調(STM)を指示する。

・ 単一焦点(Focus)だけでなく様々な Gain(Bessel や Plane 等)を一定時間毎に繰り返す触感を発生させる。

参照：https://shinolab.github.io/autd3-doc/jp/Users_Manual/API/stm.html
https://shinolab.github.io/autd3-doc/jp/Users_Manual/API/stm/gain.html

(2) 生成

○GainSTM クラス

```
pub struct GainSTM<T, C> {
    pub gains: T,
    pub config: C,
    pub option: GainSTMOption,
}
```

引数	内容
gains	Gain のリスト
config	周期またはサンプリング設定
option	オプション

○GainSTMOption クラス

```
#[repr(C)]pub struct GainSTMOption {
    pub mode: GainSTMMode,
}
```

引数	内容
mode: GainSTMMode	GainSTM モード指定

・ GainSTM では位相と振幅のデータを事前に送っておく必要があり、
送信に時間がかかることで遅延が発生する可能性がある。
この為、データ送信時間を短縮させるための GainSTM モードを指定できる。

・ GainSTM モードには、下記の何れかを指定できる。

GainSTM モード	内容
PhaseIntensityFull	振幅/位相データをすべて送信
PhaseFull	位相データのみを送信
PhaseHalf	位相を 4bit に圧縮して送信

4.11. Null クラス

(1) 概要

・ 振幅を発生させない。

・ Gain クラスを継承する。

・ Null()を送信すると振幅が発生しなくなるため、出力が停止した状態となる。

参照：https://shinolab.github.io/autd3-doc/jp/Users_Manual/API/stm.html
https://shinolab.github.io/autd3-doc/jp/Users_Manual/API/stm/gain.html

(2) 生成

○Null クラス

```
pub struct Null;
```

引数	内容
なし	

4.12. Sine クラス/SineOption クラス

(1) 概要

・ 正弦波の AM 変調を指示する。

参照：https://shinolab.github.io/autd3-doc/jp/Users_Manual/API/modulation.html
https://shinolab.github.io/autd3-doc/jp/Users_Manual/API/modulation/sine.html

(2) 生成

○Sine クラス

```
pub struct Sine<S: Into<SamplingMode> + Clone + Copy + Debug> {
    pub freq: S,
    pub option: SineOption,
}
```

引数	内容
freq	周波数
option	オプション

○SineOption クラス

```
pub struct SineOption {
    pub intensity: u8,
    pub offset: u8,
    pub phase: Angle,
    pub clamp: bool,
    pub sampling_config: SamplingConfig,
}
```

引数	内容
intensity	振幅
offset	オフセット(DC 成分)
phase	位相オフセット
clamp	true 時、波形が[0,255]範囲になる様に補正される。 false 時は、エラーとなる。
sampling_config	サンプリング設定

(3) into_nearest メソッド

```
impl Sine<Freq<f32>>
  pub const fn into_nearest(self) -> Sine<Nearest>
```

引数	内容
なし	

・ 出力不可能な周波数を指定した場合、実行時にエラーが発生する。
into_nearest()メソッドを使用すると出力可能な周波数に最も近い周波数に補正することができる。

```
Sine { freq=150.0 * Hz, option=SineOption() }.into_nearest();
```

・ 出力波形は下記となる。

$$[\ \text{intensity}/2 \times \sin(2\pi \times \text{freq} \times t + \text{phase}) + \text{offset}\ ]$$

4.13. Square クラス/SquareOption クラス

(1) 概要

・ 矩形波の AM 変調を指示する。

参照：https://shinolab.github.io/autd3-doc/jp/Users_Manual/API/modulation.html
https://shinolab.github.io/autd3-doc/jp/Users_Manual/API/modulation/square.html

(2) コンストラクタ

○Square クラス

```
pub struct Square<S: Into<SamplingMode> + Clone + Copy + Debug> {
    pub freq: S,
    pub option: SquareOption,
}
```

引数	内容
freq	周波数
option	オプション

○SquareOption クラス

```
pub struct SquareOption {
    pub low: u8,
    pub high: u8,
    pub duty: f32,
    pub sampling_config: SamplingConfig,
}
```

引数	内容
low	低レベル値
high	高レベル値
duty	デューティー比
sampling_config	サンプリング設定

・ 出力不可能な周波数を指定した場合、実行時にエラーが発生する。
into_nearest()メソッドを使用すると出力可能な周波数に最も近い周波数に補正することができる。

```
Square { freq=150.0 * Hz, option=Square () }.into_nearest();
```

4.14. Static クラス

(1) 概要

・ AM 変調を行わないことを指示する。

・ AM 変調を行わない場合、焦点位置等の変化がないと触感が発生しない。

参照：https://shinolab.github.io/autd3-doc/jp/Users_Manual/API/modulation.html
https://shinolab.github.io/autd3-doc/jp/Users_Manual/API/modulation/static.html

(2) 生成

```
pub struct Static {
    pub intensity: u8,
}
```

引数	内容
intensity	振幅

4.15. EtherCrab クラス/EtherCrabOption クラス

(1) 概要

・ EtherCrab を用いて AUTD3 デバイスと通信を行う。

・ EtherCrab クラスは Link を継承する。(Controller の open メソッドにて指定する)

参照：https://shinolab.github.io/autd3-doc/jp/Users_Manual/API/link/ethercrab.html

(2) 生成

○EtherCrab クラス

```
impl<F: Fn(usize, Status) + Send + Sync + 'static> EtherCrab<F>

  pub fn new(err_handler: F, option: impl Into<EtherCrabOptionFull>) -> Self
```

引数	内容
err_handler	エラーハンドラ
option	接続オプション

○EtherCrabOption クラス

```
pub struct EtherCrabOption {
    pub ifname: Option<String>,
    pub state_check_period: Duration,
    pub sync0_period: Duration,
    pub sync_tolerance: Duration,
    pub sync_timeout: Duration,
}
```

引数	内容
ifname	ネットワークインターフェース名
state_check_period	エラーが出ているかどうかを確認する間隔
sync0_period	同期信号の周期
sync_tolerance	同期許容レベル 初期化時、各デバイスのシステム時間差がこの値以下になるまで待機する
sync_timeout	同期タイムアウト値

・ EtherCrab 用の Link を生成する。

・ EtherCrabOption クラスは、EtherCrab クラスのインスタンス生成時に指定する。

・ メソッド default で、デフォルトのオプションを取得できる。

```
fn default() -> Self
```

・ EtherCrab を Windows 上で使用する場合、
Npcap を「WinPcap API compatible Mode」でインストールしておく必要がある。
Npcap ： https://npcap.com/

EtherCrab を Linux 上で使用する場合、root 権限が必要となるため、
sudo コマンド等にて実行させる。

```
sudo target/release/sample01_single
```

(3) Status

・ EtherCrab リンクの処理で異常等が発生した場合、
エラー情報が格納された Status を引数として、接続時に指定されたエラーハンドラを呼び出す。

・ Status の値の意味を下記に示す。

値	内容
Error = 0	SAFE-OP + ERROR 状態となった。
Lost = 1	AUTD3 デバイスがなくなった。
StateChanged = 2	SAFE-OP 状態となった。
Resumed = 4	全デバイスが OP 状態に復帰した。

以上